

Sesión 3: Estructuras de Datos

Lineales

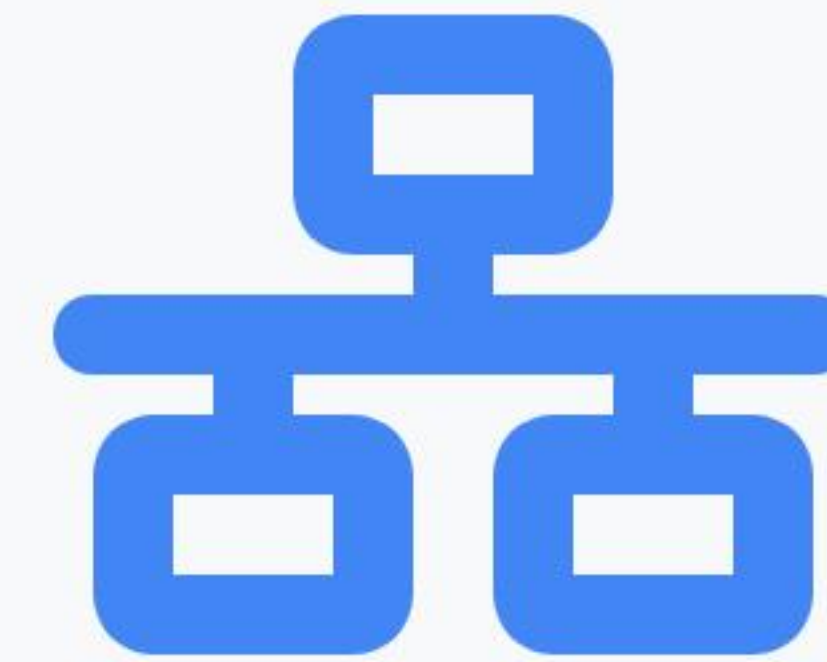
Gestión Dinámica de Memoria mediante Punteros y Nodos en C#

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

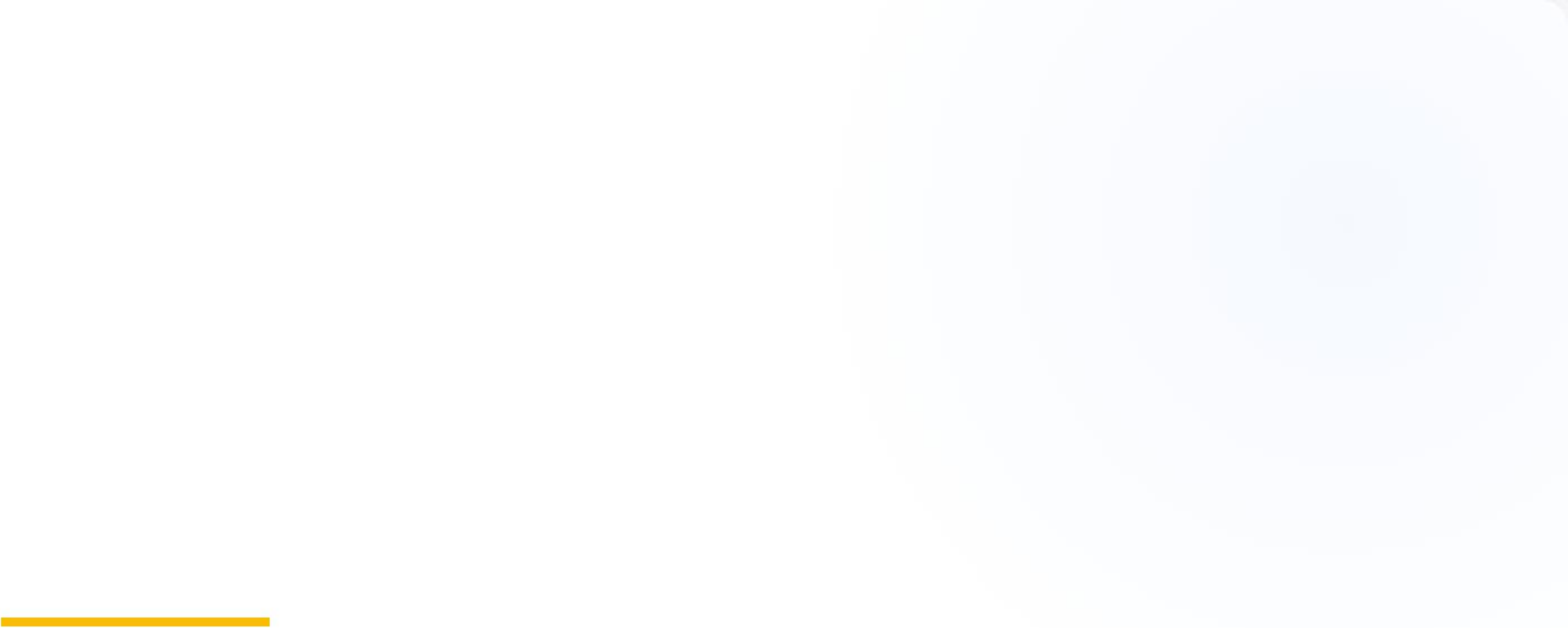
Agenda de Hoy

-  Memoria Estática vs. Memoria Dinámica
-  Listas Enlazadas Simples: Estructura de un Nodo
-  Variaciones Lineales: Listas Dobles y Circulares
-  Algoritmos CRUD de gestión interna en C#



Abstrayendo Referencias

Rompiendo la barrera de tamaño fijo impuesta por los Arrays tradicionales.



Asignación de Memoria

El núcleo del almacenamiento dinámico en sistemas computacionales

Memoria Estática vs Memoria Dinámica

Para almacenar colecciones de datos, los ingenieros debemos elegir la estrategia adecuada de reserva en hardware:

▣ Estática (Arrays)

Tamaño fijo definido en tiempo de compilación. Ocupa posiciones de memoria contiguas. Inserciones costosas de tipo **$O(n)$** debido al desplazamiento de elementos.

⚙ Dinámica (Listas)

Crece y decrece en tiempo de ejecución. Los elementos (nodos) están dispersos en la memoria Heap. Conectados de forma lógica mediante **referencias**.



Lista Enlazada Simple (Linked List)

Estructura lineal básica compuesta de eslabones referenciales

Concepto de Nodo y Referencia

Una Lista Enlazada es una secuencia de **Nodos**. Cada nodo es un objeto que contiene dos partes fundamentales de información:



- **Dato / Valor:** La información útil real que se desea almacenar (entero, string, objeto, etc.).
- **Puntero / Siguiete (Next):** Una referencia hacia el espacio de memoria que aloja al próximo nodo de la lista. El último nodo apunta siempre a `null`.

Definición de un Nodo en C#

En el paradigma de C#, un nodo se modela elegantemente mediante una clase auto-referenciada:

```
public class Nodo {  
    public int Dato { get; set; }  
    public Nodo Siguiente { get; set; } // Auto-referencia  
  
    public Nodo(int dato) {  
        this.Dato = dato;  
        this.Siguiente = null;  
    }  
}
```


Estructura Base de la Lista Simple

La clase controladora únicamente requiere un puntero de acceso principal conocido como **Cabeza (Head)**.

```
public class ListaSimple {  
    private Nodo cabeza;  
  
    public ListaSimple() {  
        cabeza = null; // Inicialmente la lista está vacía  
    }  
}
```


Operación: Insertar al Final (Append)

```
public void InsertarAlFinal(int valor) {  
    Nodo nuevoNodo = new Nodo(valor);  
    if (cabeza == null) {  
        cabeza = nuevoNodo;  
        return;  
    }  
    Nodo actual = cabeza;  
    while (actual.Siguiente != null) { // Recorrido O(n)  
        actual = actual.Siguiente;  
    }  
    actual.Siguiente = nuevoNodo;  
}
```


Operación: Recorrer y Desplegar Lista

Para imprimir o buscar datos, avanzamos de forma secuencial a través de los enlaces de los nodos:

```
public void Mostrar() {  
    Nodo actual = cabeza;  
    while (actual != null) {  
        Console.Write(actual.Dato + " -> ");  
        actual = actual.Siguiente; // Salto al siguiente bloque  
    }  
    Console.WriteLine("null");  
}
```


Operación: Eliminar un Nodo

Basta con reajustar el puntero Siguiente del nodo anterior para que "salte" el nodo que deseamos remover de la estructura.

```
// Concepto clave de desvinculación:  
Nodo anterior = BuscarNodoAnterior(valor);  
Nodo aEliminar = anterior.Siguiente;  
  
// Reajuste referencial:  
anterior.Siguiente = aEliminar.Siguiente;  
  
// En C#, el Garbage Collector limpia automáticamente la memoria de 'aEliminar'
```




Variaciones de Estructuras Lineales

Optimizando los flujos de navegación secuencial

Lista Doblemente Enlazada

Cada nodo cuenta ahora con ****dos punteros de control****, lo que permite una navegación bidireccional completa (adelante y atrás).



```
public class NodoDoble {  
    public int Dato { get; set; }  
    public NodoDoble Siguiete { get; set; }  
    public NodoDoble Anterior { get; set; } // Enlace inverso  
}
```


Lista Circular Simple

El último nodo de la lista no apunta a null; en su lugar, se conecta directamente de vuelta con el nodo **Cabeza (Head)**.

Esto genera un ciclo infinito de navegación lineal ideal para simulación de turnos, reproductores de música o buffers circulares.



Ciclo Continuo

`Ultimo.Siguiente == Cabeza`

Lista Circular Doblemente Enlazada

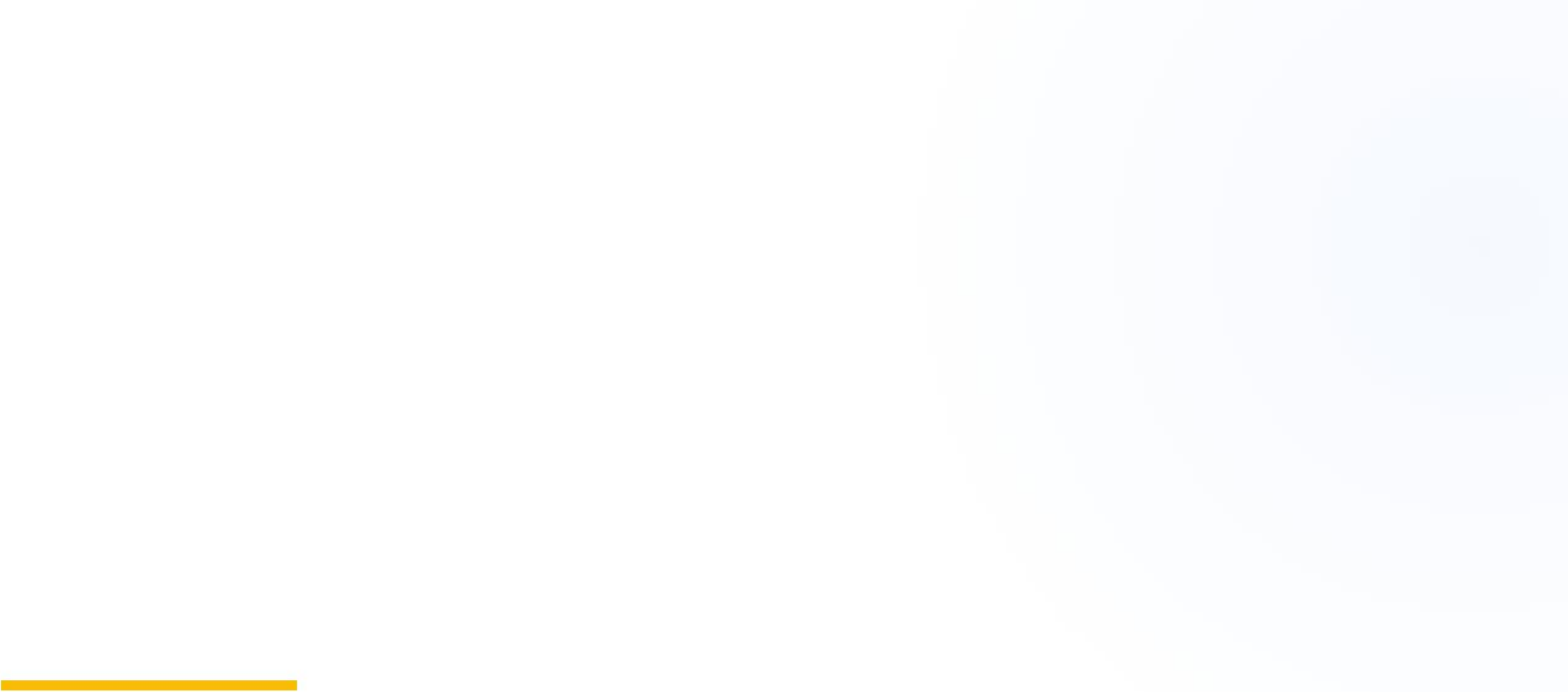
Une las dos ventajas anteriores: Navegación bidireccional y continuidad circular infinita.

Cabeza.Anterior

Te traslada de forma inmediata hacia el ****último**** nodo de la estructura (tiempo constante $O(1)$).

Ultimo.Siguiente

Te regresa inmediatamente al ****primer**** elemento de la lista sin recorrer posiciones intermedias.



Análisis de Complejidad Temporal

¿Cuándo es preferible usar colecciones enlazadas?

Matriz de Complejidad: Array vs Lista

Operación	Arreglo Fijo (Array)	Lista Enlazada Simple	Lista Doblemente Enlazada
Acceso por Índice	O(1) Directo	O(n) Secuencial	O(n) Secuencial
Insertar al Inicio	O(n) Desplazar	O(1) Directo	O(1) Directo
Insertar al Final	O(1) Si hay espacio	O(n) Recorrer completo	O(1) Si tiene puntero Cola
Eliminar Nodo	O(n) Desplazar	O(n) Búsqueda previa	O(n) Búsqueda previa



Casos de Estudio Prácticos

Aplicaciones del mundo real en ingeniería de software

¿Dónde se aplican estas Estructuras?



Historial de Cambios

El mecanismo deshacer/rehacer (Ctrl+Z / Ctrl+Y) de editores de texto se modela con Listas Dobles.



Navegadores Web

El flujo de páginas visitadas (Atrás / Adelante) guarda enlaces referenciales dinámicos.



Sistemas Operativos

La planificación de procesos por ráfagas de tiempo (Round Robin) utiliza Listas Circulares.



Buenas Prácticas y Errores Críticos

Garantizando la estabilidad referencial de punteros

Peligro Inminente: NullReferenceException

El error más común al programar estructuras en C# ocurre al intentar acceder a propiedades de un puntero que apunta a la nada.

```
// CÓDIGO INCORRECTO COMPORTAMIENTO RIESGOSO:  
while (actual.Siguiente != null) {  
    actual = actual.Siguiente;  
}  
// Si 'actual' llega a ser null, 'actual.Siguiente' romperá el sistema de forma crítica.
```

 **Regla de Oro:** Asegurar siempre la validación base de la existencia del nodo (`actual != null`) antes de consultar su enlace.

Valor de la Unidad: Responsabilidad



"Asumir las consecuencias de nuestras decisiones técnicas, entregar soluciones a tiempo y asegurar que el código desarrollado no comprometa los recursos del sistema es el reflejo de la responsabilidad profesional."

— Compromiso del Estudiante de Ingeniería

Conclusiones

Flexibilidad

Las estructuras lineales dinamizan la administración de memoria eliminando los límites estrictos de tamaño.

Punteros

Dominar los enlaces lógicos es un requisito indispensable para comprender estructuras complejas posteriores.

C# Nativo

La recolección automática de basura simplifica la liberación de nodos destruidos.

¿Dudas o Preguntas de la Sesión 3?

¡Muchas gracias por su atención!

✉ Fernando Vicente: 3021894750101@ingenieria.usac.edu.gt
💬 Módulo de consultas de Estructuras Dinámicas en UEDI